

```

import java.util.Scanner;

class PriorityScheduling {

    public static void main(String[] args) {

        System.out.println("*** Priority Scheduling (Preemptive) ***");
        System.out.print("Enter Number of Process: ");

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        // All arrays are of size n + 1 to prevent out-of-bounds
        int process[] = new int[n + 1];
        int arrivaltime[] = new int[n + 1];
        int burstTime[] = new int[n + 1];
        int completionTime[] = new int[n + 1];
        int priority[] = new int[n + 1];
        int TAT[] = new int[n + 1];
        int waitingTime[] = new int[n + 1];
        int burstTimecopy[] = new int[n + 1];

        int min = 0, count = 0;
        int temp, time = 0, end;
        double avgWT = 0, avgTAT = 0;

        for (int i = 0; i < n; i++) {
            process[i] = (i + 1);
            System.out.println("");
            System.out.print("Enter Arrival Time for processor " + (i + 1) + ": ");
            arrivaltime[i] = sc.nextInt();
            System.out.print("Enter Burst Time for processor " + (i + 1) + ": ");

```

```

burstTime[i] = sc.nextInt();

System.out.print("Enter Priority for " + (i + 1) + " process: ");

priority[i] = sc.nextInt();
}

// Sort based on arrival time and priority
for (int i = 0; i < n - 1; i++) {
    for (int j = i + 1; j < n; j++) {
        if (arrivaltime[i] > arrivaltime[j] ||
            (arrivaltime[i] == arrivaltime[j] && priority[j] > priority[i])) {

            // Swap process IDs
            temp = process[i];
            process[i] = process[j];
            process[j] = temp;

            // Swap arrival time
            temp = arrivaltime[i];
            arrivaltime[i] = arrivaltime[j];
            arrivaltime[j] = temp;

            // Swap burst time
            temp = burstTime[i];
            burstTime[i] = burstTime[j];
            burstTime[j] = temp;

            // Swap priority
            temp = priority[i];
            priority[i] = priority[j];
            priority[j] = temp;
        }
    }
}

```

```
}  
}
```

```
System.arraycopy(burstTime, 0, burstTimecopy, 0, n);
```

```
priority[n] = 999; // Sentinel value
```

```
for (time = 0; count != n; time++) {  
    min = n;  
    for (int i = 0; i < n; i++) {  
        if (arrivaltime[i] <= time && priority[i] < priority[min] && burstTime[i] > 0) {  
            min = i;  
        }  
    }  
}
```

```
burstTime[min]--;
```

```
if (burstTime[min] == 0) {  
    count++;  
    end = time + 1;  
    completionTime[min] = end;  
    waitingTime[min] = end - arrivaltime[min] - burstTimecopy[min];  
    TAT[min] = end - arrivaltime[min];  
}  
}
```

```
for (int i = 0; i < n; i++) {  
    avgTAT += TAT[i];  
    avgWT += waitingTime[i];  
}
```

```
System.out.println("\n*** Priority Scheduling (Preemptive) ***");
```

```

        System.out.println("Processor\tArrival Time\tBurst Time\tCompletion Time\tTurnaround
Time\tWaiting Time");

        System.out.println("-----");

        for (int i = 0; i < n; i++) {

            System.out.println("P" + process[i] + "\t\t" +

                arrivaltime[i] + "ms\t\t" +

                burstTimecopy[i] + "ms\t\t" +

                completionTime[i] + "ms\t\t" +

                TAT[i] + "ms\t\t" +

                waitingTime[i] + "ms");

        }

        avgWT /= n;

        avgTAT /= n;

        System.out.println("\nAverage Waiting Time: " + avgWT + "ms");

        System.out.println("Average Turnaround Time: " + avgTAT + "ms");

        sc.close();

    }

}

```